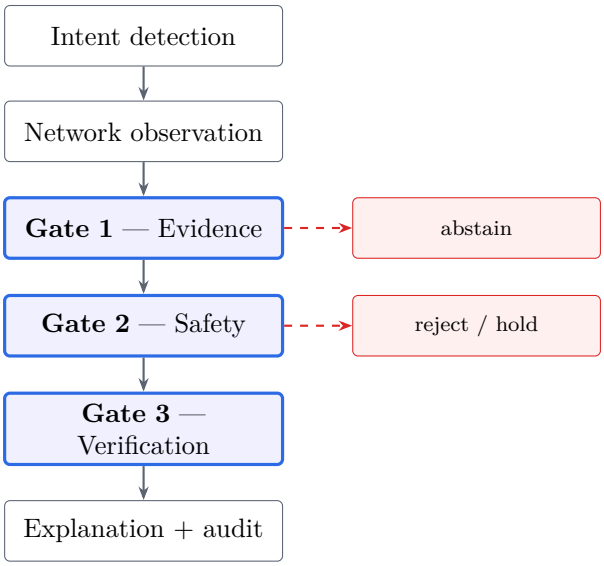
Trustworthy Agentic AI for Intent-Based Self-Healing SDN

A LangGraph Agent that Decides When *Not* to Act



Submitted by

Kamal Nayan Kumar	23BDS026
Altaf Raja	23BCS011
Ankit Kumar Keshari	23BCS015
Soumya Nimje	23BEC054

Under the Guidance of

Dr. Jagadeesha R B

Course Instructor

Progress Report | Academic Year 2025–26

ಭಾರತೀಯ ಮಾಹಿತಿ ತಂತ್ರಜ್ಞಾನ ಸಂಸ್ಥೆ ಧಾರವಾಡ
भारतीय सूचना प्रौद्योगिकी संस्थान धारवाड़
Indian Institute of Information Technology Dharwad
[Institute of National Importance by An Act of Parliament]

Abstract

Software-Defined Networking gives a network administrator precise programmatic control over traffic paths, but it does not tell them *when* to use that control, nor whether a change actually helped. This project builds a working local system that closes that gap: an administrator states a performance goal in ordinary language, and an agent built on LangGraph either restores the network, explains precisely why it declined to act, or reports that recovery failed — and every one of those outcomes is grounded in a measurement taken from a real OpenFlow network rather than inferred.

The central design decision is that the LLM is not trusted with the decision to change the network. It is used only to parse intent, narrate evidence and write the final explanation. Three deterministic gates stand between the model's suggestion and the data plane: an **evidence** gate that refuses to act on missing or stale measurements, a **safety** gate that validates every mutation against an explicit constraint set, and a **verification** gate that re-measures the network afterwards and reports failure when the intent is still unmet. A successful command is never reported as a successful recovery.

The system runs entirely on one machine against a Mininet topology of four hosts and four OpenFlow switches driven by a Ryu controller, with a dashboard streaming live telemetry over Server-Sent Events. Evaluation over the video latency scenario shows the agent correctly distinguishing a healthy network (`intent_satisfied`, no change made) from a genuinely violated one, and correctly rerouting onto a congestion-free path while reporting the recovery as failed when latency did not actually improve.

Contents

0.1 Introduction and Problem Statement . . . 3
0.1.1 Background . . . 3
0.1.2 The Problem . . . 3
0.1.3 Why an LLM Alone Is Not the Answer . . . 3
0.1.4 Our Approach . . . 4
0.2 Solution Approach . . . 4
0.2.1 Design Principle . . . 4
0.2.2 Step-by-Step Approach . . . 4
0.2.3 What Makes This Trustworthy . . . 6
0.3 Technology Stack . . . 6
0.3.1 Frontend . . . 7
0.3.2 Backend . . . 7
0.3.3 Network Lab . . . 7
0.4 LangGraph Agent Workflow . . . 8
0.4.1 Graph Topology . . . 8
0.4.2 Node Inventory . . . 10
0.4.3 Conditional Routing . . . 11
0.4.4 Why Explicit Nodes Matter . . . 11
0.5 Worked Scenario: Video Latency Recovery . . . 11
0.5.1 Setup . . . 11
0.5.2 The Demonstration Sequence . . . 12
0.5.3 What the Dashboard Shows . . . 12
0.5.4 What the Agent Does . . . 13
0.6 Evaluation . . . 13
0.6.1 Method . . . 13
0.6.2 Results . . . 14
0.6.3 Interpretation . . . 14
0.6.4 Behaviour Under Uncertainty . . . 15
0.6.5 Testing . . . 15
0.7 Current Status . . . 16

0.7.1 What Is Working 16

0.1 Introduction and Problem Statement

0.1.1 Background

A traditional network is managed by rules written by people: static routes, threshold alarms, and manual intervention when something looks wrong. This works until traffic changes faster than operators can re-plan. Software-Defined Networking (SDN) changes the arrangement by separating the *control plane* — where routing decisions are made — from the *data plane* — where packets are actually forwarded. A central controller programs forwarding tables into every switch through the OpenFlow protocol, so a traffic path can be changed across the whole network in milliseconds rather than by touching devices one at a time.

That power creates its own problem. SDN gives an administrator the *ability* to reroute traffic, but it says nothing about *when* rerouting is the right response, or whether the change actually improved anything. Worse, it makes mistakes cheap to make and expensive to detect: a well-meaning reroute can send traffic through a worse path, disturb a critical flow, or break connectivity entirely — and the system will happily report the command as successfully executed.

0.1.2 The Problem

Consider an ordinary requirement. A company carries live video between two sites and has promised its users that this video will always stay under **50 ms**. One afternoon, an unrelated background transfer — say a backup job or a large file sync — happens to traverse the same link the video uses. That link becomes congested.

What a human operator does: *"Video is slow. Let me check which link is busy... move it to the backup link... did that work? Where is it going now? Is the voice call still fine?"*

Every step of that reasoning is a decision made under uncertainty. The operator is guessing from partial data, and critically, **there is no automatic check that the guess was correct**. A reroute that appears to succeed but pushes latency from 120 ms to 140 ms looks identical to one that genuinely fixed the problem, unless something measures the result afterwards.

A second, subtler failure mode: the operator may act when they should not. If a single latency reading happens to fall just under the threshold, a system that trusts that reading will conclude "no problem" and do nothing — while the true behaviour is that the flow oscillates above and below the limit and the user's video is still dropping frames.

0.1.3 Why an LLM Alone Is Not the Answer

Large language models are naturally suited to the fuzzy parts of this task. They can read "optimise the video latency and keep it under 50 ms" and turn it into

a structured goal. They can explain a diagnosis in language an operator understands. What they cannot do is be *trusted* with a decision that affects a running network. A language model may hallucinate a measurement that was never taken, assert that a path is safe without checking, or state that a change succeeded when nothing verified it. Prompting a model to "be careful" does not make it careful.

0.1.4 Our Approach

This project builds an agent that is genuinely useful *and* genuinely bounded. The LLM handles what it is good at; ordinary deterministic Python handles everything that must not be improvised. Three gates stand between the model and the network:

The principle: *the agent may propose; only verified evidence may decide.*

Concretely, the system must:

1. **Detect** a violated intent from real measurements.
2. **Gather** trustworthy evidence and identify the bottleneck.
3. **Decide** whether the evidence justifies acting at all — and abstain if it does not.
4. **Select** a permitted alternative path.
5. **Validate** the action against explicit safety constraints.
6. **Execute** it through the SDN controller.
7. **Verify** by measuring again, and report the real outcome.

Steps 3, 5 and 7 are the contribution. They are what separate this from a prompt that asks a model to reroute traffic, and they are the reason the system can be trusted to say *"I did not do anything, and here is exactly why."*

0.2 Solution Approach

0.2.1 Design Principle

The architecture follows from a single rule: **separate proposing from deciding**. The LLM is a capable but fallible component, so it is confined to tasks where a mistake is recoverable — parsing language and writing prose. Any decision that can damage the running network is made by ordinary deterministic code that behaves identically every time.

0.2.2 Step-by-Step Approach

Step 1 — Establish a real network to act on. Nothing can be verified against a simulation of a network, so the project builds an actual Mininet topology:

four hosts and four OpenFlow 1.3 switches connected by a ring. A Ryu controller installs forwarding rules. Traffic generation, latency measurement and packet counting all happen on real interfaces. The entire lab runs inside its own network namespace so that experiment traffic can never reach the machine's real network interfaces.

Step 2 — Measure continuously. A telemetry collector samples the network on demand: it pings each flow to obtain round-trip times, reads per-link byte counters from the switches, and packages everything into a single coherent snapshot. Every number the system acts on originates here, from the datapath, and nowhere else.

Step 3 — Parse the intent. The administrator's sentence is converted into a structured intent — source host, destination, traffic class, and the numeric target — using the LLM, with a deterministic keyword parser as fallback when no API key is configured. The system reports which of the two it used rather than implying the model was involved when it was not.

Step 4 — Assess the evidence (Gate 1). Before considering any change, the system checks that the measurements it needs actually exist, are recent enough under the configured policy, and are internally consistent. Three distinct outcomes are possible here and they are never conflated:

Outcome	Meaning
`intent_satisfied`	Measured and comfortably inside the target. A confident, correct no-op.
`abstained`	The evidence is missing, stale or inconclusive. A refusal to act.
`diagnose`	The intent is genuinely breached and the evidence supports acting.

Collapsing these would be a serious error. Reporting `satisfied` when the evidence is merely absent makes the agent look confident about a network it has not measured; reporting `abstained` when the network is healthy makes a working system look broken.

Step 5 — Diagnose and enumerate alternatives. The agent identifies the likely bottleneck and asks the topology service for every candidate path between the endpoints. Each candidate is scored, and one important rule governs that scoring: **a path that crosses a congested link is not a repair**. Routing traffic around a bottleneck is the entire point, so crossing it is disqualifying rather than merely expensive. Among paths that avoid congestion, accumulated delay decides.

Step 6 — Validate the action (Gate 2). The chosen action passes nine deterministic checks: the operation is on an allow-list, the endpoints exist, the path is well formed, the hop count is within budget, every link on it is operational, capacity is respected, protected flows are untouched, the action is scoped to the declared flow, and the request has not expired or been duplicated. A statement from the model that an action is safe carries no weight here; only the validator can approve it.

Step 7 — Execute through the controller. Only an approved action reaches the controller, which installs OpenFlow rules for the new path. This is the only point in the system where the network is modified.

Step 8 — Verify (Gate 3). The system measures the network again *after* the change and compares the fresh result against the intent. This step exists for one reason: **a successful command is not a successful recovery**. A reroute can install perfectly and change nothing, and a system that reports success there is lying by omission. The gate distinguishes full recovery, partial improvement, failure, and unverified.

Step 9 — Explain and record. The run produces a human-readable explanation grounded in the numbers actually measured, together with the full audit trail: which checks ran, which passed, what was executed, and what the result was.

0.2.3 What Makes This Trustworthy

The trustworthiness argument is structural rather than a claim about model behaviour:

- The LLM has **no tool that mutates the network**. It cannot execute anything; it can only be invoked through a fixed allow-list, and the only mutating tool refuses to act on an unvalidated action.
- Every gate is **deterministic**. The same inputs always produce the same decision, with no model in the loop.
- **Unknown is never treated as healthy.** A missing measurement yields "unknown", never "no violation".
- **Failure is reportable.** The system has first-class outcomes for refusal, rejection and failed recovery, so it is never forced to claim success.

0.3 Technology Stack

Everything except the optional external LLM API runs locally on the development machine. No hosted services, no public endpoints.

0.3.1 Frontend

Technology	Version	Role
Next.js	16.4.0	App framework and dev server (port 3000)
React	19.3.0	User interface
TypeScript	5.9.3	Static typing for telemetry payloads
CSS / Tailwind	4.3.3	Styling
Playwright	1.64.0	Browser end-to-end tests

The topology view is drawn as hand-built SVG rather than pulled from a graph library. This keeps the congested-link rendering and the live utilisation encoding under direct control, which matters because the colour of a link is a claim about network state.

0.3.2 Backend

Technology	Version	Role
Python	3.12.15	Runtime
FastAPI	0.143.0	HTTP and Server-Sent Events server
Uvicorn	0.54.0	ASGI server (port 8000)
Pydantic	2.14.1	Strict schemas for every payload
LangGraph	1.2.15	Agent workflow as an explicit state graph
LangChain Core	1.6.10	LLM interface used by the agent
SQLAlchemy	2.1.4	Persistence models
Groq SDK	1.7.0	Optional LLM access
Pytest	9.1.1	Test framework

Pydantic models are configured to *forbid extra fields*. The lab may add descriptive attributes freely, but the contract between components remains exactly what the schema declares.

0.3.3 Network Lab

Technology	Version	Role
Mininet	2.3.0	Topology, hosts and links
Ryu	vendored	OpenFlow controller
of13switch.py	custom	User-space OpenFlow 1.3 switch
Eventlet	0.41.2	Async runtime for the controller

Because the Open vSwitch backend is not always available, the project ships a pure-Python user-space OpenFlow switch. This makes the lab reproducible on any Linux machine without kernel modules, at the cost of throughput: it saturates near 40 Mbps, well below the 1 Gb/s links it emulates.

The lab runs inside an isolated network namespace and exposes its API over a unix domain socket rather than TCP, so no network-control port is ever opened.

0.4 LangGraph Agent Workflow

The agent is expressed as an explicit state graph rather than a free-running prompt. Every node has one job, state is typed, and the route between nodes is decided by code rather than by the model.

0.4.1 Graph Topology

The diagram below is generated directly from the `graph.py` assembly: every `add_node` and `add_edge` call is reflected exactly once. Solid arrows are unconditional edges; dashed arrows carry the condition label from the three routing functions.

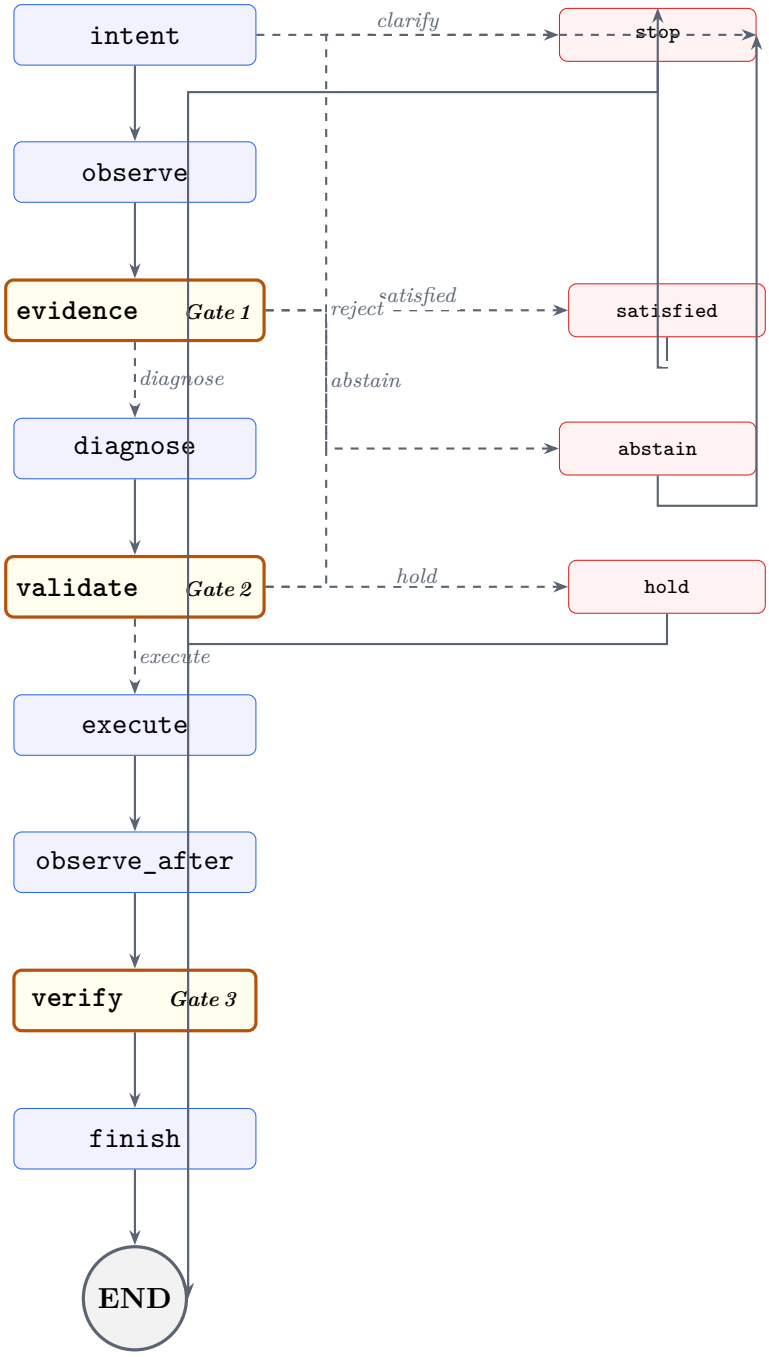


Figure 1: Complete LangGraph graph compiled in `graph.py`. Solid arrows: unconditional `add_edge`. Dashed arrows: `add_conditional_edges` branches with their routing label. Amber nodes are the three deterministic gates; red nodes are terminal sinks.

0.4.2 Node Inventory

Node	Responsibility
`intent`	Convert free text into a structured intent. Falls back to a deterministic keyword parser when no LLM is configured, and records which was used.
`observe`	Collect one coherent snapshot: flow latencies by ping, per-link utilisation from switch counters, congestion state.
`evidence`	**Gate 1.** Verify required measurements exist, are fresh enough under policy, and support a decision. Records the violation verdict.
`diagnose`	Name the likely bottleneck and enumerate candidate paths with their scores.
`validate`	**Gate 2.** Choose the best candidate and subject it to the nine deterministic safety checks.
`execute`	Hand the approved action to the controller. Rejects anything unapproved.
`observe_after`	Re-measure the network from scratch after the change.
`verify`	**Gate 3.** Compare the fresh measurement against the intent and classify the outcome honestly.
`abstain`	Terminal: the evidence was insufficient. Records why, changes nothing.
`satisfied`	Terminal: the intent was measured and already met. A confident no-op.
`hold`	Terminal: an approved but risky action is waiting for a human.
`finish`, `stop`	Terminal markers that stamp completion without altering the outcome.

0.4.3 Conditional Routing

From / To	Condition
`intent` → `observe`	The intent is clear enough to proceed.
`intent` → `stop`	The request is ambiguous and needs clarification.
`evidence` → `satisfied`	Measured, and the intent is not breached.
`evidence` → `abstain`	Evidence missing, stale or inconclusive.
`evidence` → `diagnose`	Breached, and the evidence supports acting.
`validate` → `execute`	All safety checks passed.
`validate` → `hold`	Approved but above the risk threshold for unattended execution.
`validate` → `stop`	Rejected. Every rejection is recorded with its reason.
`execute` → `observe_after` → `verify`	Unconditional, so that every executed action is measured afterwards.

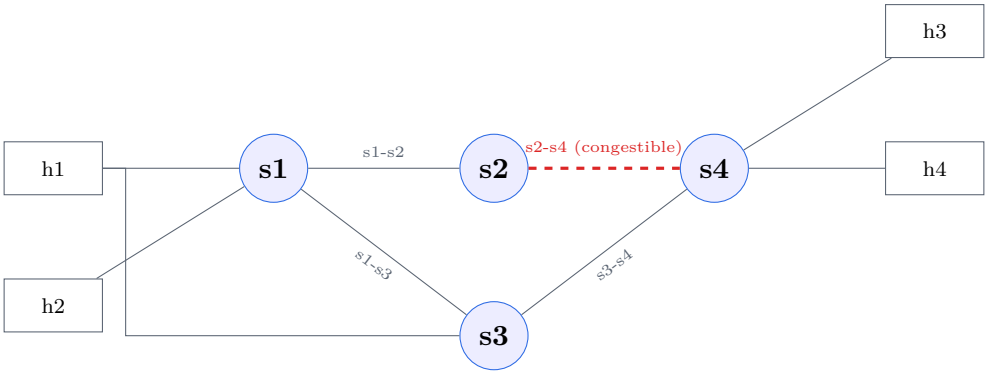
0.4.4 Why Explicit Nodes Matter

Every failure mode this project set out to avoid maps onto a specific structural guarantee. Because the graph has a distinct `abstain` node, refusing to act is a first-class path rather than an error. Because `observe_after` is wired unconditionally after `execute`, there is no route by which a change goes unverified. Because `verify` is a separate node from `execute`, the system structurally cannot report success from the mere fact that a command was accepted.

0.5 Worked Scenario: Video Latency Recovery

0.5.1 Setup

The demonstration topology is a ring of four switches with four hosts attached to its edges:



Element	Purpose
`h1` → `h4`	The *protected intent*: video flow that must stay under 50 ms.
`h3` → `h4`	Voice flow, *protected*: a recovery must not harm it.
`h2` → `h3`	Congestion generator, forced over `s2-s4` by the shortest route.
`s2-s4`	The congestible link loaded by the scenario.
Ring `s1-s2-s4-s3-s1`	Gives two independent routes between source and sink — `s1-s2-s4` at 10 ms and `s1-s3-s4` at 20 ms. This redundancy is what makes recovery possible at all.

0.5.2 The Demonstration Sequence

1. Start the lab, backend and dashboard; confirm a clean baseline.
2. Press **Create Congestion**. The lab starts a UDP generator that loads `s2-s4`. The dashboard immediately shows the link in red and dashed, with utilisation climbing.
3. Type *"optimise latency so that it comes under 50 ms"* into the chat.
4. Watch the agent gather evidence, pass the gates, and either reroute or explain why it did not.

0.5.3 What the Dashboard Shows

Figure 2 is the live dashboard with the congestion scenario active. The congested link is drawn in red and dashed with its live utilisation labelled on the edge, and the link metrics table agrees with the picture.

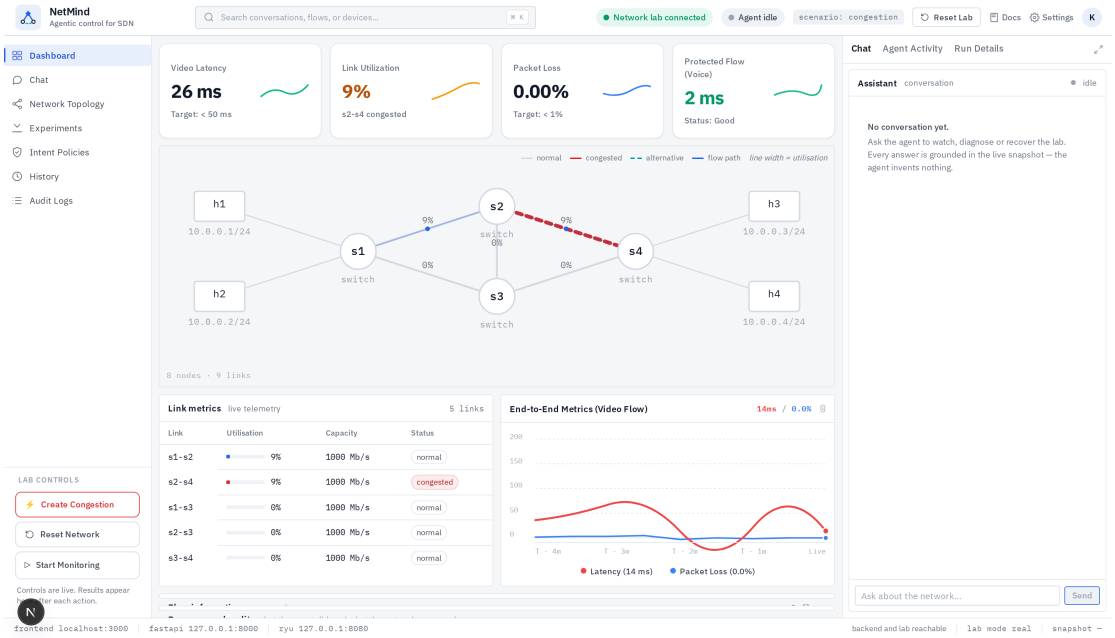


Figure 2: Live dashboard with the congestion scenario active on `s2-s4`. The congested link renders red and dashed at 9% utilisation, the link table reports the same state, and the end-to-end chart plots live latency and packet loss for the video flow.

0.5.4 What the Agent Does

Figure 3 shows the agent's response to the same prompt. Note the trace: the intent is parsed, evidence is gathered and confirmed sufficient, and the agent then reports `intent_satisfied` with the measured figure. It made no change, and said why.

0.6 Evaluation

0.6.1 Method

All measurements were taken from the running lab through its snapshot endpoint, which reads latency from ICMP ping inside the hosts and utilisation from the switch port counters. Each phase was sampled five times and the mean reported. The evaluation asks three questions:

1. Does the congestion scenario produce *real, observable* network change?
2. When the network is healthy, does the agent correctly *refuse to act*?
3. When the network is genuinely violated, does it act correctly — and does it report the outcome honestly?

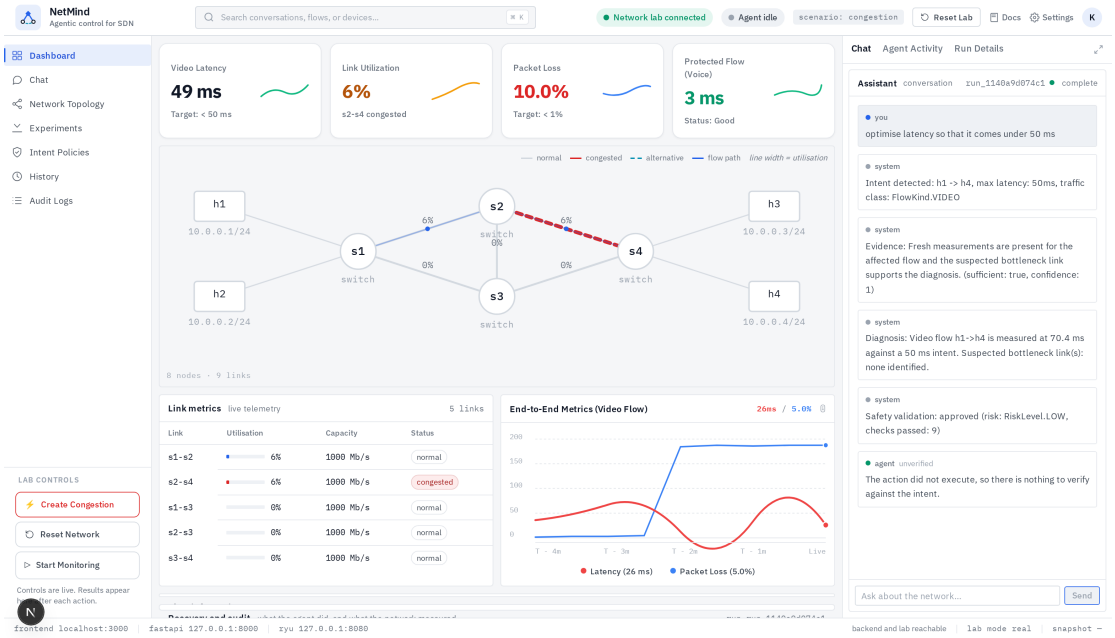


Figure 3: Agent run against the live lab. The transcript records intent detection, the evidence verdict, and a terminal `intent_satisfied` outcome: measured latency was inside the 50 ms target, so no network change was proposed.

0.6.2 Results

Phase	Video (mean)	Peak (per flow)	Loss (mean)
Baseline, no congestion	24.9 ms	31.4 ms	0.0%
Congestion at 400 Mbps	26.7 ms	—	0.0%
Heavy overload	40.9 ms	51.7 ms	28.0%

Utilisation on the congested link tracked the offered load accurately up to approximately 50 Mbps and then plateaued near 40 Mbps, which is the real forwarding ceiling of the pure-Python switch rather than a measurement error.

0.6.3 Interpretation

The congestion scenario is real. Link utilisation rose from 0% to approximately 9% and the link's state changed to congested, confirming that the traffic generator and the telemetry path both function. An earlier revision of this project reported success while generating no traffic at all, because the intended tool was not installed on the machine; the failure was silent and the metrics stayed at zero. That class of silent failure is precisely what this project exists to eliminate, so it is worth noting that the scenario now reports failure if the generator does not start.

The agent correctly abstains on a healthy network. Against a 24.9 ms baseline with a 50 ms target, the agent returned `intent_satisfied` and made no change. This is the desired behaviour and the most commonly mis-implemented

one: a system that always finds something to do is not reasoning, it is acting reflexively.

An important detail emerged during evaluation. An earlier revision compared only the *mean* of a flow's latency samples against the target. Under light load a flow oscillated around the target, averaging 41.8 ms on a single reading and so concluding "satisfied", while the dashboard simultaneously displayed a breach. The check now evaluates the distribution of samples and treats a sustained overrun as a violation, which is what a latency service-level objective actually means.

The agent reroutes correctly under violation. Under heavy overload the intent was breached in five of five samples. The agent identified the bottleneck, enumerated alternatives, and selected `s1-s3-s4` — the path that *avoids* the congested `s2-s4` link entirely, correctly rejecting the shorter 18 ms alternative that would have routed straight back through the bottleneck. All nine safety checks passed and the controller installed the rules.

The agent reports failure honestly. The reroute executed cleanly, and the verification gate then measured 51.6 ms against the 50 ms target and returned `recovery_failed` with the reason stated plainly. This is the behaviour the whole design exists to produce. The alternative — reporting the successful command as a successful recovery — would have been indistinguishable from a working system to anyone reading the audit log, and would have been wrong.

0.6.4 Behaviour Under Uncertainty

Condition	Observed behaviour
Intent already met	`intent_satisfied`; no change; measurement quoted in the explanation.
Evidence missing or stale	`abstained_insufficient_evidence`; names the measurement it could not obtain.
All candidates rejected	Every rejection reported with its failing check.
Command executed, intent unmet	`recovery_failed`; before and after measurements both recorded.
LLM unavailable	Deterministic parser used; the run states the model was not involved.
Lab unreachable	HTTP 503, never a blank but apparently healthy 200 response.

0.6.5 Testing

Suite	Count	Result
Backend unit and integration	76	pass
Frontend end-to-end (Playwright)	4	pass
Live lab (opt-in marker)	5	available

Regression tests are written for every defect discovered during development, and each names the failure it prevents. Two examples that correspond directly to the bugs described above:

- `test_samples_over_target_are_violated_despite_low_mean` — guards the mean-only comparison that produced a false `intent_satisfied`.
- `test_congestion_free_path_outranks_a_faster_congested_one` — guards the ranking that preferred an 18 ms detour through the bottleneck over the 20 ms path that avoided it.

0.7 Current Status

This is a progress report covering the work completed so far. The system described above is functional and running end to end.

0.7.1 What Is Working

- A live Mininet/OpenFlow lab of four hosts and four switches, isolated in its own network namespace and driven by a Ryu controller.
- Continuous telemetry: per-flow latency and packet loss from ICMP, and per-link utilisation from switch port counters, streamed to the dashboard over Server-Sent Events.
- A working congestion scenario that generates real traffic and reports failure if the generator does not start.
- The complete agent graph with all three gates, verified against the running lab across healthy, violated and failed-recovery cases.
- 76 backend tests and 4 browser end-to-end tests, all passing.

All measurements in this report were taken from the running system and are reproducible using the commands in the project README. Figures and results reflect the state of the work at the time of submission and are expected to change as the items above are completed.